BASICS OF JAVA PROGRAMMING
BPLCK105C
MODULE 2

1. Basic Arithmetic Operators

a) Definition:

Arithmetic operators perform mathematical operations like addition, subtraction, multiplication, and division.

b) Operators and Their Use:

Operator	Description	Example
+	Addition	`a + b` (Adds `a` and `b`)
-	Subtraction	`a - b` (Subtracts `b` from `a`)
*	Multiplication	`a * b` (Multiplies `a` and `b`)
/	Division	`a / b` (Divides `a` by `b`)
%	Modulus	`a % b` (Remainder when `a` is divided by `b`)

Example:

```
class ArithmeticExample {
    public static void main(String[] args) {
        int a = 10, b = 5;
        System.out.println("Addition: " + (a + b));  // 15
        System.out.println("Subtraction: " + (a - b)); // 5
        System.out.println("Multiplication: " + (a * b)); // 50
        System.out.println("Division: " + (a / b)); // 2
        System.out.println("Modulus: " + (a % b)); // 0
    }
}
```

2. Increment, Decrement, and Ternary Operator

a) Increment (++) and Decrement (--)

- Pre-Increment (++a): Increases a before using it.
- Post-Increment (a++): Uses a first, then increases.

b) Ternary Operator (? :)

- A shortcut for if-else conditions.
- Syntax: variable = (condition) ? value1 : value2;

c) Example:

```
class Operators {
    public static void main(String[] args) {
```

```
        int x = 10;
        int y = ++x; // Pre-increment
        int z = (x > y) ? x : y; // Ternary operator
        System.out.println("y: " + y);
        System.out.println("z: " + z);
    }
}
```

3. Bitwise Operators in Java

a) Definition:

Bitwise operators **manipulate individual bits** of integers and perform operations like AND, OR, XOR, and shifts.

b) Types of Bitwise Operators:

Operator	Description	Example
`&`	Bitwise AND – Returns `1` if both bits are `1`	`a & b`
` ` `	` ` `	Bitwise OR – Returns `1` if any bit is `1`
`^`	Bitwise XOR – Returns `1` if bits are different	`a ^ b`
`~`	Bitwise NOT – Inverts all bits	`~a`
`<<`	Left Shift – Moves bits left (multiplies by 2)	`a << 2`
`>>`	Right Shift – Moves bits right (divides by 2)	`a >> 2`
`>>>`	Unsigned Right Shift – Shifts right, fills with zeros	`a >>> 2`

c) Example: Bitwise AND, OR, XOR

```
class BitwiseExample {
    public static void main(String args[]) {
        int a = 5, b = 3; // Binary: 5 = 0101, 3 = 0011
        System.out.println("a & b = " + (a & b)); // AND (0001 = 1)
        System.out.println("a | b = " + (a | b)); // OR (0111 = 7)
        System.out.println("a ^ b = " + (a ^ b)); // XOR (0110 = 6)
    }
}
```

Output:

```
a & b = 1
a | b = 7
a ^ b = 6
```

d) Example: Bitwise Shift Operators

```
class ShiftExample {
```

```
    public static void main(String args[]) {
        int x = 8; // Binary: 1000
        System.out.println("x << 1 = " + (x << 1)); // Left shift (10000 = 16)
        System.out.println("x >> 1 = " + (x >> 1)); // Right shift (0100 = 4)
    }
}
```

Output:

```
x << 1 = 16
x >> 1 = 4
```

e) Explanation:

Bitwise operators perform operations on binary digits instead of whole numbers.

Left shift (<<) multiplies by 2, while right shift (>>) divides by 2.

Bitwise NOT (~) inverts all bits, flipping 1 to 0 and vice versa.

4. Relational Operators

a) Definition:

Relational operators compare two values and return true or false.

b) Operators and Their Use:

Operator	Description	Example
==	Equal to	a==b
!=	Not equal to	a!=b
>	Gretter than	a>b
<	Less than	a<b
>=	Gretter than or equal to	a>=b
<=	Less than or equal to	a<=b

c) Example:

```
class RelationalExample {
    public static void main(String[] args) {
        int x = 10, y = 5;
        System.out.println("x > y: " + (x > y));
    }
```

```
}
```

5. Types of if Statements in Java

a) Definition:

The if statement is a **control flow statement** used to execute a block of code **only if a specified condition is true**.

b) Types of if Statements:

1. **Simple if Statement** – Executes a block of code **if the condition is true**.
    - **Syntax:**

```
if (condition) {
   // Code executes if condition is true
}
```

Example:

```
int x = 10;
if (x > 5) {
   System.out.println("x is greater than 5");
}
```

2. **if-else Statement** – Executes **one block if true**, another if false.

Syntax:

```
if (condition) {
   // Executes if true
} else {
   // Executes if false
}
```

Example:

```
int num = 10;
if (num % 2 == 0) {
   System.out.println("Even Number");
} else {
   System.out.println("Odd Number");
}
```

3. **Nested if Statement** – An if statement **inside another if statement**.

Syntax:

```
if (condition1) {
    if (condition2) {
        // Executes if both conditions are true
    }
}
```

Example:

```
int age = 20;
if (age > 18) {
    if (age < 60) {
        System.out.println("Eligible for work");
    }
}
```

6. Switch Statement in Java

a) Definition:
The switch statement is a multi-way branch statement that allows a variable to be tested for equality against multiple values, making it an alternative to long if-else-if chains.

b) Syntax:

```
switch (expression) {
    case value1:
        // Statements
        break;
    case value2:
        // Statements
        break;
    default:
        // Default case if no match found
}
```

c) Example: Basic switch Statement

```
class SwitchExample {
    public static void main(String args[]) {
        int day = 3;
        switch (day) {
            case 1:
                System.out.println("Monday");
                break;
            case 2:
                System.out.println("Tuesday");
                break;
            case 3:
                System.out.println("Wednesday");
                break;
            default:
                System.out.println("Invalid Day");
        }
    }
}
```

Output:

Wednesday

7. Iteration Statements in Java

Iteration statements (loops) allow a block of code to execute repeatedly until a condition is met

1. **while Loop**
Definition:
Executes a block as long as the condition is true.
Syntax & Example:

```
int i = 5;
while (i > 0) {
    System.out.println("Countdown: " + i);
    i--;
}
```

Output:

```
Countdown: 5
Countdown: 4
Countdown: 3
Countdown: 2
Countdown: 1
```

2. do-while Loop
Definition:
Executes the block **at least once**, even if the condition is false.
Syntax & Example:

```
int i = 5;
do {
    System.out.println("Number: " + i);
    i--;
} while (i > 0);
```

Output:

```
Number: 5
Number: 4
Number: 3
Number: 2
Number: 1
```

3. for Loop
Definition:
Executes a block **a fixed number of times**.
Syntax & Example:

```
for (int i = 1; i <= 5; i++) {
    System.out.println("Iteration: " + i);
}
```

Output:

```
Iteration: 1
Iteration: 2
Iteration: 3
Iteration: 4
Iteration: 5
```

8. Break , Continue & Return Statements in Java OR Jump statements in java

1. break Statement
Definition:
The break statement immediately exits a loop or a switch statement, transferring control to the next statement outside the loop.
Syntax & Example:

```
for (int i = 0; i < 10; i++) {
    if (i == 5) break; // Exits loop when i is 5
    System.out.println(i);
}
```

Output :
0
1
2
3
4

Key Points:
✅ **Used to exit for, while, do-while, and switch statements.**
✅ **Stops execution of the loop immediately when a condition is met.**
✅ **Can be labeled to break out of nested loops.**

2. continue Statement
Definition:
The continue statement skips the current iteration of a loop and moves to the next iteration.
Syntax & Example:

```
for (int i = 0; i < 5; i++) {
    if (i == 2) continue; // Skips iteration when i is 2
    System.out.println(i);
}
```

Output:
0
1
3
4

Key Points:

✅ **Used to skip specific iterations of a loop.**
✅ **Moves control directly to the next iteration.**
✅ **Can be labelled to continue an outer loop in nested structures.**

3. return Statement
Definition:
The return statement exits from a method and optionally returns a value to the calling method.
Syntax & Example:

```
class ReturnExample {
    static int sum(int a, int b) {
        return a + b; // Returns sum to caller
    }
    public static void main(String args[]) {
        int result = sum(10, 5);
        System.out.println("Sum: " + result);
    }
}
```

Output:
Sum: 15

Use Cases:
✅ **Exiting a method when a certain condition is met.**
✅ **Returning values from methods.**
✅ **Used in recursion for returning computed results.**